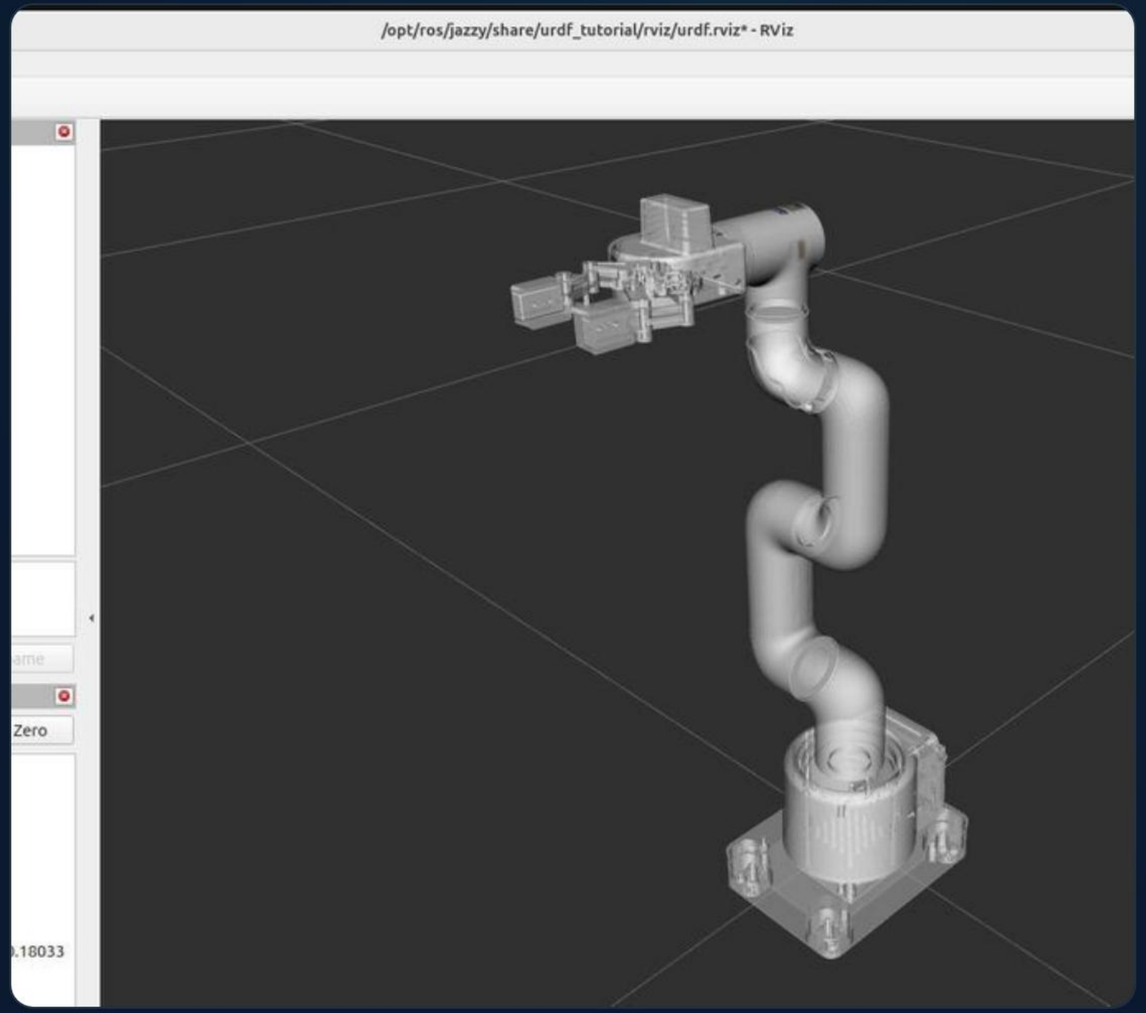
ROS 2 입문 세미나

ROS 2의 시간, 파일 시스템, 빌드, 패키지

분산 로봇 시스템에서 꼭 알아야 하는 4가지 기반 개념

→ 시간 동기화 → 코드 구조 → 빌드 → 배포/재사용

>_ 예시 환경: Humble 기반 워크스페이스



왜 이 4개를 한 번에 이해해야 하는가

시간 (Time)

메시지가 **언제** 생성되었는가 (데이터 신뢰성 확보)

파일 시스템

패키지가 **어디** 있는가 (구조적 일관성 유지)

빌드 (Build)

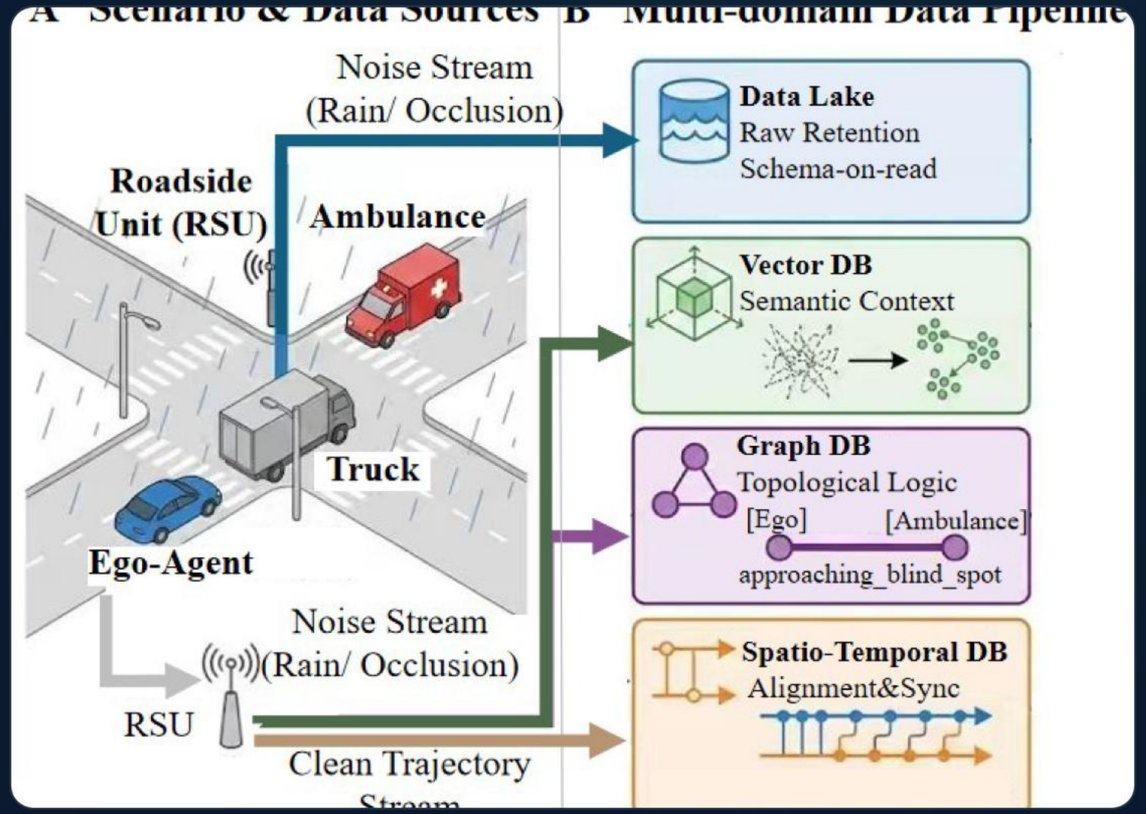
의존성을 **어떻게** 해석하는가 (실행 파일 생성)

패키지 파일

도구가 **무엇을** 알아야 하는가 (메타데이터 제공)

왜 시간(timestamp)이 그렇게 중요한가

- 센서 데이터는 시간 순서가 맞아야 의미가 있음
- 여러 노드 간 통신에서 publish 시각이 중요
- Header의 stamp, frame_id가 자주 함께 사용됨
- 예시: 카메라 30Hz, LiDAR 10Hz, IMU 200Hz



Clock와 Time은 무엇이 다른가

Clock

시간 소스에 접근하는 창구. 시스템 시계 혹은 시뮬레이션 시계 자체를 의미합니다.

Time

특정 시점. 시계가 가리키는 한 순간을 나타내는 데이터입니다.

Duration

두 시점의 차이. 시간의 간격(델타 타임)을 의미합니다.

System Time, ROS Time, Steady Time

System Time

운영체제 시스템 시간 (UTC 기반)

ROS Time

/clock 기반 시뮬레이션/로그 재생 시간

Steady Time

단조 증가. 하드웨어 타임아웃에 최적화

시뮬레이션 시간은 어떻게 동작하는가

- **use_sim_time=true**이면 ROS Time 활성화
- /clock이 publish되면 시스템 시간 대신 사용
- /clock을 못 받으면 시간이 0에 머무를 수 있음
- 시간 점프, 일시정지, 빠른 재생 가능

ROS 2의 Time API: Time, Duration, Rate, Timer

- **Time:** 현재 시각 획득, 시점 비교, 덧셈/뺄셈 연산 수행
- **Duration:** 경과 시간 측정 및 하드웨어 타임아웃 계산
- **Rate:** 루프 내에서 특정 주기를 일정하게 유지
- **Timer:** 노드 내에서 주기적으로 콜백 함수를 실행하는 표준 방식

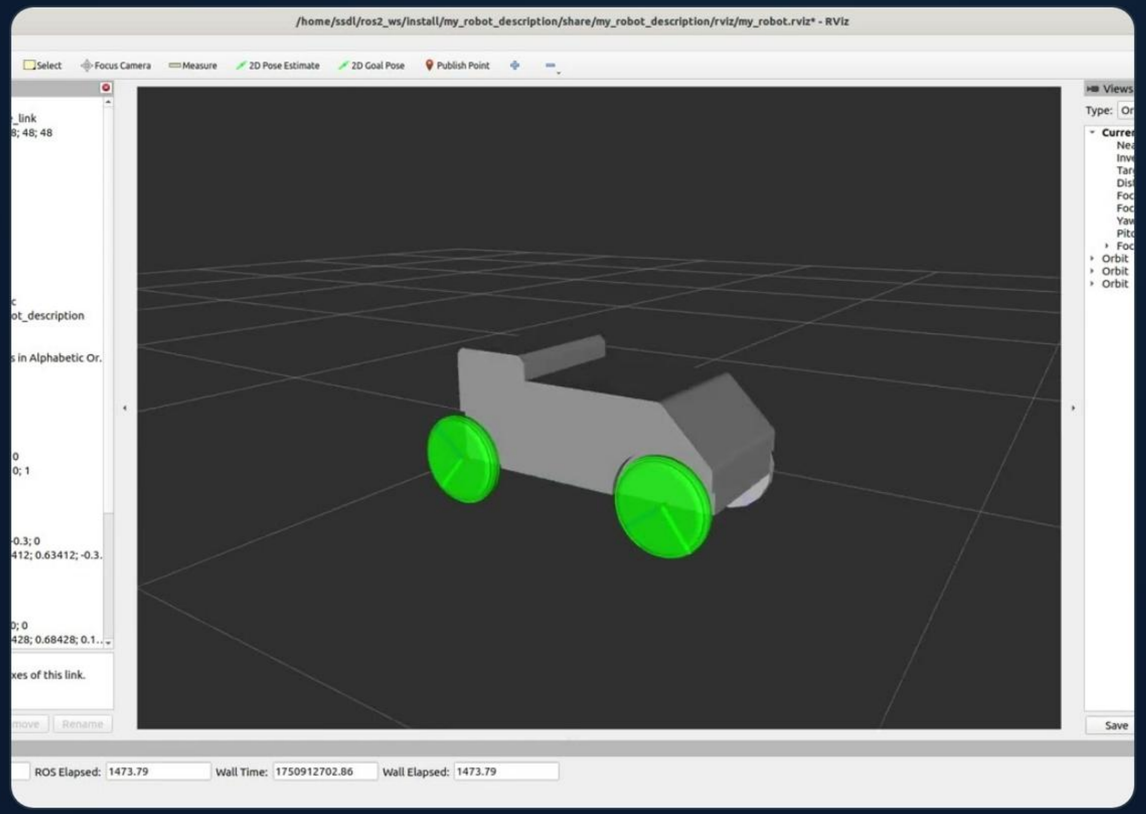
예제 코드로 보는 시간 연산

```
// C++ Example
rclcpp::Time now = node->now();

// 5초 경과 여부 체크
if ((now - past).seconds() > 5.0) {
  RCLCPP_INFO(logger, "Over 5s!");
}

// 미래 timestamp 생성 (1초 뒤)
msg.stamp = now + rclcpp::Duration(1, 0);
```

- **now - past**: 경과 시간 계산
- **now + duration**: 미래 timestamp 생성
- 메시지의 stamp 필드에 시간 정보를 주입하여 신뢰성 있는 센서 퓨전 구현

ROS 2 파일 시스템은 왜 규칙적이어야 하는가

- 패키지 및 실행 파일 검색의 **자동화**
- 패키지 루트는 **package.xml**로 식별
- 한 워크스페이스에 여러 패키지 공존 가능
- 일반적으로 **src/** 폴더 하위에 패키지 생성



패키지, 메타패키지, variant를 어떻게 설명할까

Package

ROS 2 응용 프로그램의 최소 논리 단위입니다.

Metapackage

여러 관련 패키지를 묶어 관리하는 개념입니다. (예: Nav2)

실무에서는 **exec_depend**를 통해 관련 패키지를 묶는 방식을 주로 사용합니다.

패키지는 설치해서 쓸까, 소스로 빌드할까

바이너리 설치 (apt)

검증된 패키지를 빠르게 설치하여 바로 실행할 때 사용합니다.

```
sudo apt install ros-humble-...
```

소스 빌드 (colcon)

코드 수정, 디버깅, 최신 기능이 필요할 때 활용합니다.

```
git clone ... && colcon build
```

/opt/ros와 ros2_ws는 어떤 차이인가

Underlay (/opt/ros)

시스템 수준에서 설치된 ROS 2 핵심 및 기본 라이브러리 경로입니다.

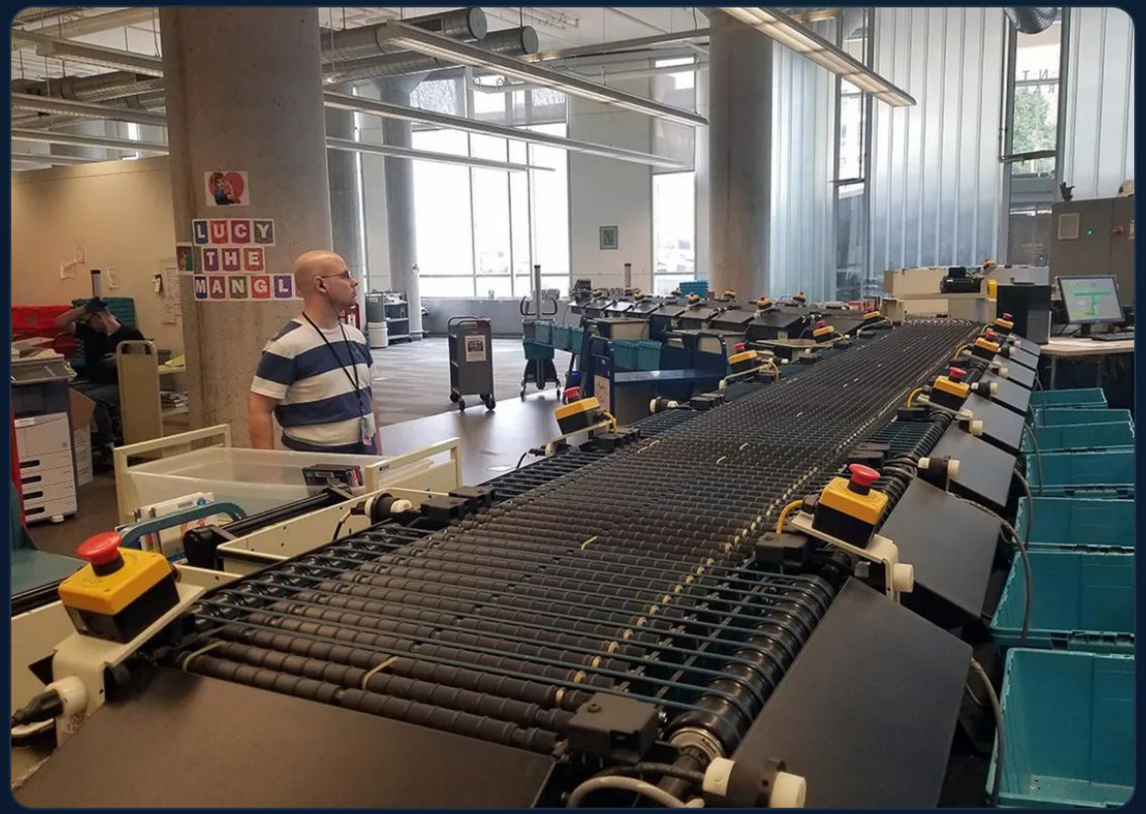
Overlay (~/ros2_ws)

사용자가 개발 중인 패키지들이 위치하는 작업 워크스페이스입니다.

local_setup.bash는 현재 Overlay만, setup.bash는 Underlay까지 포함합니다.

ROS 2 빌드는 한 층이 아니다

- **Build Tool:** CMake (C++), setuptools (Python)
- **Build Helper:** ament (ROS 2 전용 편의 도구)
- **Meta-build Tool:** colcon (전체 의존성 관리)
- 의존성 그래프의 기준은 **package.xml**



ament_cmake와 ament_python은 무엇이 다른가

ament_cmake

C++ 개발을 위한 빌드 타입입니다. CMake의 강력한 기능을 기반으로 ROS 2 매크로가 결합되어 있습니다.

ament_python

Python 개발을 위한 빌드 타입입니다. setuptools를 사용하여 스크립트 기반 설치를 수행합니다.

패키지 생성과 빌드 실전 명령어

```
# 패키지 생성
ros2 pkg create --build-type ament_cmake --node-name my_node my_package

# 전체 빌드 및 심볼릭 링크 설치
colcon build --symlink-install --packages-up-to my_package
```

- 🔗 **--symlink-install:** 스크립트 수정 시 재빌드 없이 즉시 반영
- ↑ **--packages-up-to:** 지정 패키지와 상위 의존성까지 빌드

협업에서 꼭 나오는 3가지 보조 도구

vcstool

여러 저장소의 특정 버전을 .repos 파일로 일괄 관리

rosdep

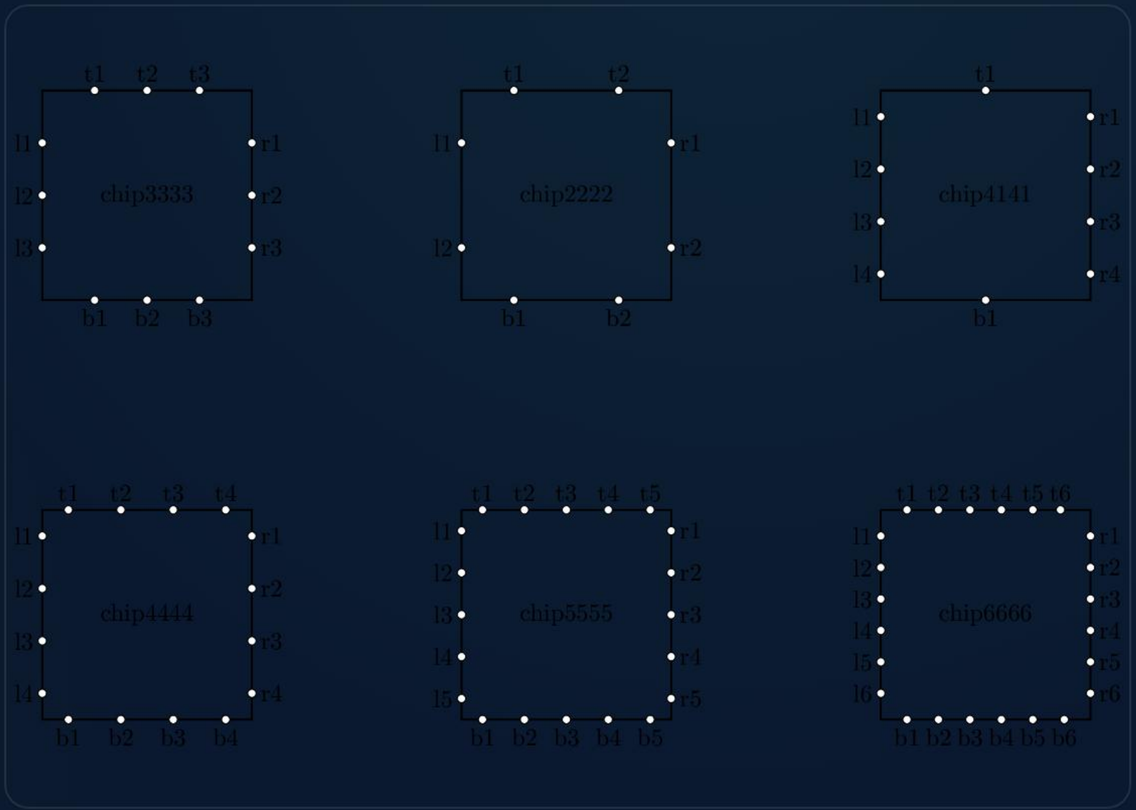
package.xml 기반으로 시스템 의존성을 자동 설치

bloom

패키지를 apt로 배포할 수 있는 바이너리 릴리스 지원

패키지 안에는 어떤 파일이 들어가는가

- **package.xml:** 모든 패키지의 필수 Manifest
- **CMakeLists.txt:** C++ 빌드 규칙
- **setup.py/cfg:** Python 설치 규칙
- README, LICENSE, CHANGELOG 등



package.xml은 도구가 읽는 패키지 설명서다

```
sensor_fusion
Dev
ament_cmake
rclcpp
launch_ros
```

- : build + build_export + exec 의존성 축약형
- 릴리스 도구와 빌드 도구가 이 파일을 통해 패키지 정보를 해석합니다.

C++ 패키지의 빌드 규칙: CMakeLists.txt

```
find_package(rclcpp REQUIRED)
add_executable(talker src/talker.cpp)
ament_target_dependencies(talker rclcpp)
install(TARGETS talker DESTINATION lib/${PROJECT_NAME})
ament_package()
```

- **find_package:** 의존 패키지 탐색
- **ament_target_dependencies:** ROS 2 패키지 의존성 연결
- **install:** 바이너리 설치 경로 지정

Python 패키지의 설치 규칙: setup.py

```
entry_points={
  'console_scripts': [
    'talker = my_pkg.talker:main',
  ],
}
# setup.cfg 에서 script-dir를
# lib/package_name 으로 조정 필요
```

- **entry_points:** 실행 명령과 Python 함수 연결
- **data_files:** package.xml과 리소스 설치 설정

실무에서는 문서/메타데이터가 더 중요할 때가 있다

plugin.xml

RQt 등 시스템에 플러그인 존재를 알림

CHANGELOG.rst

버전별 변경 이력 관리 (릴리스 필수)

LICENSE

재사용을 위한 법적 근거 명시

README.md

개발자를 위한 설치 및 사용 가이드

CONCLUSION

ROS 2 Core Pipeline

- **src** 폴더에 패키지 생성 및 **package.xml** 의존성 명시
- 언어에 맞는 빌드 파일(CMake/setup) 작성
- **rosdep**으로 의존성 설치 후 **colcon build**
- **source** 명령어로 Overlay 환경 활성화

Any Questions?

THANK YOU FOR YOUR ATTENTION

Image Sources

https://automaticaddison.com/wp-content/uploads/2024/11/mycobot280-elephant-robotics-rviz-ros2.jpg

Source: automaticaddison.com

---

https://www.preprints.org/frontend/picture/ms_xml/manuscript/907f6359e009478937d317c0fde7628b/preprints-202340-g004.png

Source: www.preprints.org

---

https://i.ytimg.com/vi/pNCqkosEHhQ/maxresdefault.jpg

Source: medium.com

---

https://cdn.prod.website-files.com/5dfcf65cd9df4b6e43be0ece/6436268ec6fc9e579e9fabc8_SeveralBins_20171011%20-%20web.webp

Source: www.tech-logic.com

---

https://lcapy.readthedocs.io/en/latest/_images/chips4.png

Source: lcapy.readthedocs.io